

Corn Trait Extraction Using 2D Projected Images from 3D Point Cloud Images

Sudan Baral
PhD Student, BAE
sudanb@ksu.edu

Abstract - This study investigates whether three-dimensional geometric measurements of field-grown maize plants, stem length, internode lengths, and leaf arc lengths, can be regressed directly from two-dimensional RGB images derived from terrestrial laser scan (TLS) point clouds. Using the MaizeField3D dataset, comprising 1,045 high-quality TLS point clouds of field-grown maize (*Zea mays*), 520 segmented plant pairs were projected into 39,000 images across 75 azimuth angles and used to train and evaluate three convolutional neural network architectures: a pretrained EfficientNet-B3, a pretrained ResNet-34, and a custom CNN trained from scratch. All models share an identical three-layer MLP regression head producing a 32-slot output vector of physical measurements in meters. Results show that all three models yield negative R^2 across every output slot under both single-view and multi-view evaluation protocols, performing worse than a mean predictor baseline. A Wilcoxon signed-rank test ($n = 78$, $\alpha = 0.05$) confirms that all three models perform statistically significantly worse than the mean predictor baseline ($p < 0.001$ for all models), and pairwise comparisons between architectures reveal no statistically significant differences ($p > 0.28$). The uniform failure across architectures with differing capacity and pretraining indicates that the bottleneck is the irreversible loss of depth and spatial layout during 3D-to-2D projection, rather than any property of the model. These findings establish a formally defensible negative result and motivate depth-aware or direct point cloud approaches for future plant phenotyping pipelines.

INTRODUCTION

Agriculture is a rapidly evolving domain that increasingly leverages sophisticated sensors and hardware for the collection of information-rich datasets. However, high-throughput morphological phenotyping faces a critical bottleneck: data scarcity caused by the seasonal nature of crop growth and weather-dependent collection cycles. To mitigate these restrictions, researchers must identify novel techniques for generating and utilizing

existing high-fidelity data, such as terrestrial laser scan (TLS) point cloud datasets. Plant phenotyping serves as an essential tool for providing the spatio-temporal structural information required by breeders and farmers to track vital features. In the context of maize (*Zea mays*), morphological traits such as stem internode length and leaf length are direct indicators of a plant's structural integrity and physical strength. While 3D point clouds provide a robust representation of these traits, processing such data is computationally intensive and difficult to deploy in real-time agricultural robotics. This study investigates the feasibility of regressing these 3D spatial measurements from 2D RGB images synthesized from the MaizeField3D point cloud dataset. The technical objective is to evaluate a multi-view 2D projection pipeline that utilizes a virtual camera model to simulate field deployment scenarios. By adopting a 3D-to-2D projection strategy, this work achieves a 75-fold gain in annotation efficiency, as per-leaf 3D labels propagate automatically across all virtual views, bypassing the intensive manual labor typical of field-grown maize imagery. To establish a performance baseline, this research compares three distinct convolutional neural network (CNN) architectures: a pre-trained ResNet-34, a pre-trained EfficientNet-B3, and a custom CNN trained from scratch. The objective is to determine if standard ImageNet-derived weights are transferable to sparse geometric projections or if the task requires a custom architectural prior. Performance is rigorously quantified using Mean Absolute Error (MAE) and Coefficient of Determination (R^2) metrics, validated by the Wilcoxon Signed-Rank Test to ensure statistical significance. The MaizeField3D dataset [1], introduced by the Baskar group at Iowa State University, provides 1,045 high-quality TLS point clouds of field-grown maize from a diverse genetic panel.

Background and Related Works

High-throughput plant phenotyping has increasingly relied on three-dimensional sensing technologies to overcome the fundamental limitations of 2D image-based approaches. Convolutional neural networks have become the dominant tool for image-based phenotyping tasks

including trait regression, organ detection, and stress classification [2]. Jiang and Li [2] provide a comprehensive review of CNN architectures applied to plant phenotyping demonstrating state-of-the-art performance across classification, detection, and segmentation tasks when models pretrained on ImageNet are fine-tuned on plant imagery. Multitask learning extensions of pretrained ResNet backbones have further been shown to improve trait estimation by sharing representations across related targets such as leaf count, projected leaf area, and genotype classification [3].

The transition from 2D to 3D phenotyping has been motivated by the structural limitations of single-view imagery. Terrestrial laser scanning provides dense, geometrically accurate point clouds that preserve leaf architecture, plant volume, and spatial arrangement information that 2D images fundamentally cannot capture. A widely adopted strategy for leveraging 2D deep learning infrastructure on 3D data is projection-based processing, in which point clouds are rendered onto 2D planes and processed by CNNs, with results optionally reprojected back into 3D space [4]. Schunck et al. [5] demonstrated multi-view semantic labeling of plant point clouds by fusing CNN predictions from multiple 2D projections, achieving strong segmentation results on TLS data of sugar beet. However, projection-based approaches are consistently noted to suffer from depth ambiguity and occlusion artifacts that degrade geometric fidelity [4][6].

Sun et al. [6] provide a systematic review of deep learning methods for 3D plant phenomics, categorising approaches into projection-based, voxel-based, and point-based methods. They note that the first generation of 3D phenotyping studies used indirect 2D-to-3D reprojection frameworks, while more recent work operates directly on raw point cloud representations. The limitations of projection-based methods for metric tasks are highlighted, as projection discards depth and causes irrecoverable occlusion for densely foliated plants such as maize. Direct point cloud methods, exemplified by PointNet [7], which introduced a permutation-invariant architecture that consumes raw point sets without voxelisation or rendering, offer a principled alternative that avoids these failure modes entirely. Subsequent work such as the 2D-to-3D reprojection comparison by Meier et al. [8] showed that for plant organ segmentation, projection-based pipelines leveraging large 2D pretraining can compete with purpose-built 3D networks, but their evaluation was limited to segmentation labels rather than continuous metric regression. The present study is, to our knowledge, the first systematic evaluation of whether direct metric regression of continuous 3D plant measurements from 2D projected images is feasible, and provides a formally tested negative result that motivates future work in depth-aware or point-cloud-native regression pipelines.

METHODOLOGY

A. Dataset

All experiments use the MaizeField3D dataset, a collection of terrestrial laser scan (TLS) point clouds of field-grown maize (*Zea mays*). The original dataset includes 1,045 raw point cloud files and 520 segmented plants with individually labeled leaves and stems, collected with a TLS scanner. A subsampled version with 100K points per plant was used for this work. TLS was chosen over real RGB imagery because field-grown maize is densely self-occluding, and per-leaf 3D labeling done once in the point cloud propagates automatically to all projected views, providing a $75\times$ annotation efficiency gain over manual image-level annotation.

The dataset was split into 520 plant pairs (raw + segmented). Plants are partitioned at the plant level, rather than the image level, to prevent data leakage, since all 75 views of a single plant share the same 3D geometry. The split is 364 plants (27,300 images) for training, 78 plants (5,850 images) for validation, and 78 plants (5,850 images) for test, using a fixed random seed (3).

TABLE 1. DATASET SPLIT.

Splits	Plants	Images(x75)
Train	364	27300
Test	78	5850
Val	78	5850

B. 3D-to-2D Projection

Training images are produced by projecting point clouds onto a virtual camera using perspective projection, simulating an Intel RealSense D457 sensor (horizontal FOV 90° , vertical FOV 65° , 640×640 px, $f_x \approx 614.86$ px, $f_y \approx 593.34$ px). Real camera intrinsics were used to make the synthetic images geometrically consistent with a plausible field deployment scenario.

Raw PLY files from the TLS are in scanner space with Z pointing downward. Before projection, both the raw and segmented clouds are centered by subtracting the raw cloud centroid, and the Z axis is flipped so that +Z points upward. The camera is moved to 75 different positions around the plant using spherical coordinates: five azimuths (15° , 60° , 135° , 225° , 315°), five elevations (-30° , -15° , 0° , $+15^\circ$, $+30^\circ$), and three radial distances (1.0 m, 1.25 m, 1.5 m). For each viewpoint, an extrinsic matrix is built from the camera position so the optical axis points at the plant origin, and world points are projected to pixel coordinates using the pinhole model. Occlusion is handled by depth-sorting (painter's algorithm, far-to-near).

Each viewpoint produces three registered output images: (i) an RGB image from raw-cloud colors, (ii) a segmented color image from the labeled cloud, and (iii) a uint16 instance mask where pixel values encode instance

identity (0 = background, 1 = stem, 2...N = leaf1...leafN, ordered bottom-to-top by mean Z height). To fill gaps caused by sparse point projections at larger distances, a morphological densification step propagates colors and instance IDs to empty pixels via nearest-neighbor lookup after morphological closing.

C. Stem length, internode length and leaf length

The morphological information from corn plant is extracted as ground truth for the model to be trained. Stem internode keypoints are located via k-d tree nearest neighbour search. This identifies where the leaves are attached to the leaf(node). The leaves are ordered top to bottom by their mean Z co-ordinate. The length of stem is calculated as cumulative Euclidian Arc length from base to its adjacent node (1 internode), and so on until reaching the top.

Similarly, each leaf's point cloud is first cleaned using a statistical outlier removal (SOR) filter to discard stray scan noise, then subsampled to at most 2,000 points (seeded RNG for reproducibility) to keep graph operations tractable. A k-nearest-neighbour graph (k=8) is built over the remaining points by connecting each point to its 8 closest neighbours with edges weighted by 3D Euclidean distance, after which a minimum spanning tree (MST) is extracted, a cycle-free subgraph that connects all points with minimum total edge length, naturally tracing the leaf's skeleton along its midrib and blade. The arc length of the leaf is then taken as the diameter of this MST, i.e. the longest shortest path between any two nodes in the tree, which for a leaf almost always runs from base to tip and accumulates edge lengths along the curve rather than measuring straight-line distance, correctly capturing the true length of a bowed or curved leaf.

The per plant dictionary is saved as labels.json which stores stem length, internode lengths and leaf lengths in meter.

D. Models and Architectures

To determine the type of baseline to use, GenAI was used. As per its suggestion, a pre-trained vanilla model with resnet backbone works like a good baseline as it is highly used and cited model. In our case, since the type of dataset we are using is very unique, we decided to use another neural net model too, efficientnet_b3. With two pretrained baselines, EfficientNet_B3 and ResNet-34 architecture. A custom CNN model is built with model training done from scratch. Here, the concept is to verify the validity of dataset more than just the model. Both pretrained models are imported from torchvision library of pytorch.

The models share the same MLP regression head which outputs a vector with 32 indexes where 0 is the stem length and followed by internode length and leaf lengths. The baseline models differ in their encoder part which are

responsible for extracting the features and information of the plant organs.

MLP-head-architecture:

Let x represent the feature vector from the encoder ($x \in \mathbb{R}^{d_{in}}$). The MLP head performs the following transformation to produce the prediction $\hat{y}$ ($\hat{y} \in \mathbb{R}^N$):

$$\hat{y} = \sigma(W3 \cdot \sigma(BN2(W2 \cdot \sigma(BN1(W1x + b1)) + b2)) + b3)$$

Notation Definitions:

- x : Input feature vector from the encoder.
- $\hat{y}$: Output prediction vector representing the N plant traits.
- $W1, W2, W3$: Learnable weight matrices for the linear layers.
- $b1, b2, b3$: Learnable bias vectors.
- σ (sigma): The Rectified Linear Unit (ReLU) activation, defined as $\max(0, z)$.
- BN: Batch Normalization, which normalizes the activations to maintain stable gradients during training.

Physical Constraint: The final activation σ is applied to the output layer to enforce the constraint $\hat{y} \geq 0$. This ensures that the model cannot predict negative values for physical traits like node length or stem diameter, aligning the model's output space with biological reality.

ResNet-34 Encoder architecture (with pretrained weights):

1. Feature Extraction: The input image x is processed by the ResNet-34 encoder (E) and Global Average Pooling (P) to produce a fixed-length feature vector g : $g = P(E(x))$, where $g \in \mathbb{R}^{512}$
2. Residual Logic: Each encoder block maintains structural information via skip connections: $z_{out} = \sigma(F(z) + z)$ (where $F(z)$ is the convolutional mapping and $+z$ is the identity shortcut)
3. Regression Mapping: The vector g is passed through the MLP head to produce the trait predictions $\hat{y}$: $\hat{y} = \sigma(W3 \cdot \sigma(BN2(W2 \cdot \sigma(BN1(W1 \cdot g + b1)) + b2)) + b3)$

Definitions:

- σ : ReLU activation, $\max(0, z)$, ensuring $\hat{y} \geq 0$.
- BN: Batch Normalization for training stability.
- W, b : Learnable weights and biases.
- $\hat{y}$: Output vector of N plant trait measurements (e.g., stem/internode lengths).

EfficientNet_B3 encoder architecture (with pretrained weights):

1. Feature Extraction: The input image x is processed by the EfficientNet-B3 encoder (E) and Global Average Pooling (P) to produce a

compressed feature vector $g: g = P(E(x))$, where $g \in \mathbb{R}^{1536}$

2. Efficiency Logic: The encoder utilizes Mobile Inverted Bottleneck (MBConv) blocks, which combine depthwise separable convolutions with Squeeze-and-Excitation (SE) attention: $z_{out} = SE(Depthwise_Conv(Inverted_Bottleneck(z)))$
3. Regression Mapping: The 1536-dimensional vector g is passed through the MLP head to produce the trait predictions $y_{hat}: y_{hat} = \sigma(W3 \cdot \sigma(BN2(W2 \cdot \sigma(BN1(W1 \cdot g + b1)) + b2)) + b3)$

Definitions:

- σ : ReLU activation, $\max(0, z)$, enforcing the physical constraint $y_{hat} \geq 0$.
- BN: Batch Normalization for internal covariate shift reduction.
- W, b : Learnable weights and biases.
- y_{hat} : Output vector of N plant trait measurements (e.g., stem/internode lengths).

CNN architecture (Not pre-trained)

□ Feature Extraction: The input x is processed through an initial 7×7 convolution and max pooling, followed by a sequence of 8 residual convolution blocks to produce the feature vector $g: g = P(Encoder(x))$, where $g \in \mathbb{R}^{1536}$

□ Residual Connection: Each internal block ($ConvBlock$) utilizes a skip connection to prevent signal degradation during training from scratch: $z_{out} = \sigma(F(z) + S(z))$ (where $F(z)$ represents the double-convolution path and $S(z)$ is the identity or downsampling shortcut)

□ Regression Mapping: The final feature vector g is mapped to trait predictions y_{hat} via the MLP head: $y_{hat} = \sigma(W3 \cdot \sigma(BN2(W2 \cdot \sigma(BN1(W1 \cdot g + b1)) + b2)) + b3)$

E. Masked Mean Squared Error (Loss Function)

The loss L calculates the squared error only for valid nodes, utilizing a binary mask m to handle missing data in the point cloud projections:

$$L = \sum ((y_{hat} - y_{true})^2 \cdot m) / \sum (m)$$

- m : Binary mask where 1 indicates a present trait and 0 indicates missing data.
- Σ : Summation over all batch elements and target features.
- Utility: This prevents "penalizing" the model for failing to predict traits that were not visible or segmented in the original 3D point cloud.

Definitions:

- σ : ReLU activation, $\max(0, z)$.
- y_{hat} : Predicted trait vector.
- y_{true} : Ground truth trait vector.
- BN: Batch Normalization for training stability.

EXPERIMENTS AND EVALUATION

A. Model Training

After the dataset were prepared, and models were chosen, model training was initialized. Key parameters that were kept same throughout the models were Epoch size: 30 (with exception of CNN, 40 epochs, as it was untrained), batch size 16 to keep parameters consistent with each other and give better apples to apples comparison. AdamW optimizer was used as a standard. The models have patience of 15 epochs, the model stops if there is no decrement in val_loss . If there is decrement, the patience counter resets to 0, else it keeps adding up, and even after 15 epochs, the latest val loss is still higher than the last continuous 15 val loss, the model training stops.

To make training streamlined, all the training configs were launched using a single `train_all` python node. Best checkpoint is saved when val_loss improves.

TABLE 2. BEST VAL_LOSS FOR EACH MODEL.

Model	Epochs	Best Val_loss
EfficientNet_B3	18	0.0763(ep 3)
RestNet-34	19	0.0736(ep 4)
Custom CNN	32	0.0745 (ep 12)

B. Evaluation metrics

First, evaluation protocol was developed. We choose Single-View(SV) and Multiview(MV) evaluations. Single view is when each image that was projected, ie. 75 different views are evaluated. For 78 plants that's 5850 predictions. Similarly, for MV evaluation, the 75 predictions of each plant is averaged out into single prediction that gives prediction of 1 plant per evaluation.

The overall performance of the models were decided based on their MAE, RMSE and R^2 scores for the regression output.

Mean Absolute Error (MAE) is the average absolute deviation between predicted value $\hat{y}_i$ and ground truth y_i across n valid samples:

$$MAE = (1/n) \cdot \sum_i |\hat{y}_i - y_i|$$

MAE is expressed in meters and is the primary metric for comparing models, as it is robust to outliers and directly interpretable as average measurement error.

Root Mean Squared Error (RMSE) penalises large deviations more heavily than MAE by squaring the residuals before averaging:

$$RMSE = \sqrt{(1/n) \cdot \sum_i (\hat{y}_i - y_i)^2}$$

A large gap between RMSE and MAE indicates the presence of a small number of severe prediction errors.

Coefficient of Determination (R^2) measures the proportion of variance in the ground truth explained by the model, relative to a mean predictor $\bar{y}$:

$$R^2 = 1 - [\sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2]$$

$R^2 = 1$ indicates perfect prediction. $R^2 = 0$ indicates the model performs identically to always predicting the mean. $R^2 < 0$ indicates the model performs worse than a mean predictor — the model's predictions are farther from the truth than simply guessing the average. All three models in this study return $R^2 < 0$ across virtually every output slot,

Model	SV MAE (m)	SV RMSE (m)	SV R^2	MV MAE (m)	MV RMS E (m)	M V R^2
EfficientNet-B3	0.1861	0.2782	< 0	0.1834	0.2738	< 0
ResNet-34	0.1848	0.2758	< 0	0.1823	0.2720	< 0
Custom CNN	0.1857	0.2790	< 0	0.1849	0.2778	< 0

which is the primary indicator of failure.

C. Statistical Analysis

To determine scientifically whether the untrained CNN model did better than the baseline trained models, following hypothesis are tested.

H_0 : $\text{Med}(d_i) = 0$ — model error is not different from the mean predictor

H_1 : $\text{Med}(d_i) \neq 0$ — model error is significantly different from the mean predictor

Where $d_i = \text{MAE}_{\text{base}} - \text{MAE}_{\text{model}}$ is the per-plant difference in absolute error. Positive d_i indicates the model outperforms the baseline for plant i ; negative d_i indicates it performs worse.

Where $d_i = \text{MAE}_{\text{base}} - \text{MAE}_{\text{model}}$

This is the per plant difference in absolute error between mean predictor baseline and the model. Positive d_i means that the model outperforms the baseline for plant i . As suggested by Statistical test suggestion by course instructor and GenAI suggestions, Wilcoxon Signed-Rank Test was chosen.

For each plant i , the per-plant MAE is computed over all valid (non-padded) slots:

$$\text{MAE}_i = (1/|V_i|) \cdot \sum_{j \in V_i} |\hat{y}_{ij} - y_{ij}|$$

where V_i is the set of valid (non-zero-padded) slots for plant i .

The mean predictor baseline predicts the per-slot mean across all 78 test plants:

$$\bar{y}_j = (1/78) \cdot \sum_i y_{ij}$$

$$\text{MAE}_{\text{base}} = (1/|V_i|) \cdot \sum_{j \in V_i} |\bar{y}_j - y_{ij}|$$

The signed difference for each plant is then:

$$d_i = \text{MAE}_{\text{base}} - \text{MAE}_{\text{model}}$$

The test proceeds in three steps:

Step 1 — Rank absolute differences: Assign rank $r_i = \text{rank}(|d_i|)$ to each plant, discarding ties and zero differences.

Step 2 — Compute the test statistic:

$$W^+ = \sum_i: d_i > 0 \ r_i \quad W^- = \sum_i: d_i < 0 \ r_i$$

Step 3 — Interpret: Under H_0 with $n = 78$, the expected value of W^+ is:

$$E[W^+] = n(n+1)/4 = 78 \times 79 / 4 = 1540.5$$

$$z = (W^+ - n(n+1)/4) / \sqrt{(n(n+1)(2n+1)/24)}$$

$$p = 2 \cdot \min(\Phi(z), 1 - \Phi(z)) \text{ [two-sided]}$$

where Φ is the standard normal CDF. A W^+ well below 1540.5 indicates that the signed ranks are dominated by W^- — the model performs worse than the baseline for the majority of plants. Because z is strongly negative for all three models, $\Phi(z) \approx 0$, and the two-sided p-value is $p \approx 2 \cdot \Phi(z) \ll 0.05$, yielding rejection of H_0 . The significance level is set at $\alpha = 0.05$.

DISCUSSION AND FAILURE MODES

A. Result interpretation

TABLE 4. WILCOXON SIGNED-RANK TEST: EACH MODEL VS. MEAN PREDICTOR BASELINE (ONE-SIDED, $\alpha = 0.05$)

Model	Model MAE (m)	Baseline MAE (m)	W statistic	z	p-value
EfficientNet-B3	0.1825	0.1726	834.0	-3.52	0.0004
ResNet-34	0.1815	0.1726	745.0	-3.96	0.00007
Custom CNN	0.1835	0.1726	771.0	-3.82	0.00001

All three models return W^+ statistics (745–834) far below the null expectation of 1540.5, with strongly negative z-scores and two-sided p-values well below $\alpha = 0.05$. The signed ranks are dominated by W^- , confirming that all three models produce higher error than the mean predictor for the majority of the 78 test plants. H_0 is rejected for all three models: each model performs statistically significantly worse than the mean predictor baseline.

Pairwise Wilcoxon tests (two-sided) confirm that no model is statistically distinguishable from any other:

Pairwise Wilcoxon tests (two-sided) confirm that no model is statistically distinguishable from any other:

TABLE 5. PAIRWISE WILCOXON SIGNED-RANK TEST BETWEEN MODELS (TWO-SIDED, $\alpha = 0.05$)

Comparison	W statistic	p-value	Significant?
ResNet-34 vs Custom CNN	1327.0	0.2876	No
EfficientNet-B3 vs Custom CNN	1361.0	0.3713	No
EfficientNet-B3 vs ResNet-34	1488.0	0.7937	No

No pairwise comparison reaches significance (all $p > 0.28$). A pretrained 12M-parameter EfficientNet-B3, a pretrained ResNet-34, and a Custom CNN trained from scratch are statistically indistinguishable in their prediction error. Taken together, the results establish a formally defensible negative finding: the failure is attributable to the task formulation itself, specifically, the irreversible loss of depth and spatial layout information during 3D-to-2D projection, rather than to any property of model architecture or capacity.

B. Failure modes

The results of the test was not optimistic, revealing multiple failure modes, which provide a great insight on how to deal with such unique dataset. The main failure started with loss of depth after projection. The images were projected with fixed intrinsic and extrinsic, however, after projection they lost the depth. This causes the model to confuse when the same plant is in different orientation, in tracking the same pixel. This is a fundamentally dataset problem.

Similarly, the different baselines and untrained CNN model show comparable similarity in the errors which shows, that the imagenet trained weights are useless, since there is a domain shift in the type of dataset and also, it was meant for segmentation, but we did regression, so the pretrained weights were as good as no weights, hence the similar result to the untrained CNN.

CONCLUSION AND FUTURE WORK

A. Summary

In summary, it was deduced that using projected images for corn plant trait extraction might not be feasible. If anything, more robust dataset, and better training head should be designed which can take in rgb images as well as depth information, such that deducing intermodal distances is possible. Our proposed model not significantly better than the baselines, the baselines and the models were simply not able to work with the dataset we provided. Our original intention was to make trait extraction possible using monocular cameras (cannot get depth information). This would have been great for deployment and real-time trait extraction

B. Future work

For future work, I would recommend to assess dataset feasibility before working with models. In context of corn plant phenotyping, our future work will be:

- Use 3D point cloud images/depth images
- Use better keypoint extraction algorithm.
- Use real corn image dataset if possible.

REFERENCES

- [1] E. Kimara, M. Hadadi, J. Godbersen, A. Balu, T. Jubery, Y. Li, A. Krishnamurthy, P. S. Schnable, and B. Ganapathysubramanian, "MaizeField3D: A curated 3D point cloud and procedural model dataset of field-grown maize from a diversity panel," arXiv:2503.07813, 2025.
- [2] Y. Jiang and C. Li, "Convolutional neural networks for image-based high-throughput plant phenotyping: A review," Plant Phenomics, vol. 2020, p. 4152816, 2020, doi: 10.34133/2020/4152816.
- [3] A. Dobrescu, M. V. Giuffrida, and S. A. Tsaftaris, "Doing more with less: A multitask deep learning approach in plant phenotyping," Front. Plant Sci., vol. 11, p. 141, 2020, doi: 10.3389/fpls.2020.00141.
- [4] X. Guo et al., "Delving into the potential of deep learning algorithms for point cloud segmentation at organ level in plant phenotyping," Remote Sensing, vol. 16, no. 17, p. 3290, 2024, doi: 10.3390/rs16173290.
- [5] B. Japes, J. Mack, F. Rist, K. Herzog, R. Töpfer, and V. Steinhage, "Multi-view semantic labeling of 3D point clouds for automated plant phenotyping," arXiv:1805.03994, 2018.
- [6] S. Jin, D. Li, T. Yun et al., "Deep learning for three-dimensional (3D) plant phenomics," Plant Phenomics, vol. 7, no. 4, p. 100107, 2025, doi: 10.1016/j.plaphe.2025.100107.
- [7] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 652–66.